# AI - Assisted Coding-Assignment -17.2

**HTNO:2303A52228**
**Batch:37**

**Lab 17 – AI for Data Processing: Data Cleaning and Preprocessing Scripts.**

**Task Description-1**
Student Academic Data Cleaning.
**Expected Output:**
A cleaned and encoded Pandas DataFrame suitable for academic performance analysis.
**Prompt:**
#generate a Python script that cleans and prepares a student academic dataset for analysis.Handle missing values in marks, attendance, and department.Remove duplicate student records based on student ID.
#Standardize text fields such as department names.Encode categorical variables like gender and department.create a student dataset
**CODE AND OUTPUT :**

lab17.py

hpc15 > lab17.py > ...

```python
#Standardize text fields such as department names.Encode categorical variables like gender and department.
#create a student dataset
import pandas as pd
data = pd.read_csv("students.csv")
data['marks'] = data['marks'].fillna(data['marks'].mean())
data['attendance'] = data['attendance'].fillna(data['attendance'].median())
data['department'] = data['department'].fillna(data['department'].mode()[0])
data = data.drop_duplicates(subset='student_id')
data['department'] = data['department'].str.lower().str.strip()
data['department'] = data['department'].replace({
    'computer science': 'cse',
    'cse ': 'cse',
    'ece ': 'ece'
})
data['gender'] = data['gender'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['department'])
data['marks'] = (data['marks'] - data['marks'].min()) / (data['marks'].max() - data['marks'].min())
data['attendance'] = (data['attendance'] - data['attendance'].min()) / (data['attendance'].max() - data['at
print(data)
data.to_csv("cleaned_students.csv", index=False)
```

```
(venv) PS C:\Users\ajayr\Documents\use\OneDrive\Desktop\Abhi\3_2> & c:\Users\ajayr\Documents\use\OneDrive\
\python.exe c:/Users/ajayr/Documents/use/OneDrive/Desktop/Abhi/3_2/hpc15/lab17.py
   student_id   name  gender     marks  attendance  department_cse  department_ece  department_mech
0         101   Amit       0  0.562500    0.666667            True           False            False
1         102   Sara       1  0.875000    0.333333           False            True            False
2         103   John       0  0.578125    0.533333            True           False            False
3         104  Priya       1  0.125000    0.600000            True           False            False
4         105  Rahul       0  0.750000    0.800000           False            True            False
5         106    Anu       1  1.000000    1.000000           False           False             True
6         107  David       0  0.000000    0.000000            True           False            False
7         108  Meena       1  0.578125    0.466667            True           False            False
8         109  Kiran       0  0.500000    0.600000           False            True            False
9         110   Ravi       0  0.812500    0.733333           False           False             True
```

**Explanation:**

This experiment focuses on preparing a student academic dataset so that it can be used effectively for analysis or machine learning. First, the dataset is loaded into Python using the Pandas library, which makes it easy to work with structured data. Since real-world data often contains missing values, these are handled carefully—marks are replaced with the average value, attendance with the median, and missing department entries with the most common department. This ensures the dataset is complete and reliable. Next, duplicate student records are removed using the student ID so that each student appears only once.

**Task Description-2**
Financial Transaction Data Preprocessing.
**Expected Output:**
A transformed dataset with normalized values and newly generated features.
**Prompt:**
Generate a Python script using Pandas to preprocess financial transaction data. Convert the transaction date column to datetime format, extract month and year as new features, handle extreme values in the transaction amount using appropriate techniques, and normalize the amount column.
**CODE AND OUTPUT :**

hpc15 > lab17.py > ...

```python
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
transactions = pd.read_csv("transactions.csv")
transactions['transaction_date'] = pd.to_datetime(transactions['transaction_date'])
transactions['year'] = transactions['transaction_date'].dt.year
transactions['month'] = transactions['transaction_date'].dt.month
Q1 = transactions['amount'].quantile(0.25)
Q3 = transactions['amount'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
median_value = transactions['amount'].median()
transactions['amount'] = transactions['amount'].apply(
    lambda x: median_value if x < lower_bound or x > upper_bound else x
)
scaler = MinMaxScaler()
transactions['amount'] = scaler.fit_transform(transactions[['amount']])
print(transactions.head())
transactions.to_csv("cleaned_transactions.csv", index=False)
```

```
(venv) PS C:\Users\ajayr\Documents\use\OneDrive\Desktop\Abhi\3_2> & c:\Users\ajayr\Documents\us
\python.exe c:/Users/ajayr/Documents/use/OneDrive/Desktop/Abhi/3_2/hpc15/lab17.py
   transaction_id  customer_id transaction_date    amount transaction_type  year  month
0               1          101       2024-01-15  0.010152           credit  2024      1
1               2          102       2024-02-20  0.060914            debit  2024      2
2               3          103       2024-03-05  1.000000           credit  2024      3
3               4          104       2024-01-25  0.000000            debit  2024      1
4               5          105       2024-02-10  0.020305           credit  2024      2
```

**Explanation:**
In this experiment, financial transaction data is preprocessed to make it suitable for

analysis and machine learning. First, the transaction date column is converted into a proper datetime format so that time-based operations can be performed easily. From this, new features such as transaction year and month are extracted, which help in understanding trends over time. Next, the transaction amount is analyzed to detect extreme values or outliers using the IQR method, and such unusual values are replaced with the median to avoid distortion in analysis.

**Task Description-3**
Environmental Sensor Data Preparation.
**Expected Output:**
A fully preprocessed dataset ready for predictive modeling tasks.
**Prompt:**
Generate a Python script to preprocess environmental sensor data by handling missing values, encoding categorical variables, applying standard scaling to numerical features, and splitting the dataset into training and testing sets.
**CODE AND OUTPUT:**

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
data = pd.read_csv("sensor_data.csv")
data['temperature'] = data['temperature'].fillna(data['temperature'].mean())
data['humidity'] = data['humidity'].fillna(data['humidity'].median())
data['air_quality_index'] = data['air_quality_index'].fillna(data['air_quality_index'].mean())
data['sensor_status'] = data['sensor_status'].map({'active': 1, 'inactive': 0})
scaler = StandardScaler()
data[['temperature', 'humidity', 'air_quality_index']] = scaler.fit_transform(
    data[['temperature', 'humidity', 'air_quality_index']]
)
X = data.drop('sensor_status', axis=1)
y = data['sensor_status']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
print(X_train.head())
print(y_train.head())
```

```
   sensor_id  temperature  humidity  air_quality_index
5          6     0.731925  0.442817           0.873806
3          4    -1.707825 -0.189779           0.000000
4          5     1.707825  0.864548           1.609642
6          7     0.000000  1.286279           0.137969
7          8    -1.219875  0.231952          -0.597867
5    1
3    1
4    0
6    1
7    0
```

**Explanation:**
In this experiment, environmental sensor data is prepared for predictive modeling by performing several preprocessing steps. Initially, missing values in temperature,

humidity, and air quality index are handled using appropriate statistical methods such as mean and median to ensure data completeness. Next, the categorical field sensor status is converted into numerical form so that it can be used in machine learning models. After that, numerical features are scaled using standard scaling, which transforms the data to have a mean of zero and a standard deviation of one, ensuring all features contribute equally.

**Task Description-4**
Real-Time Application: Smart City Traffic Data Cleaning.
**Expected Output:**
A cleaned and normalized traffic dataset with an accompanying data-quality improvement summary.
**Prompt:**
Generate a Python script to clean smart city traffic data by handling missing values, removing duplicates, standardizing text fields, normalizing numerical features, and comparing dataset quality before and after preprocessing.
**CODE AND OUTPUT :**

```
import pandas as pd
traffic = pd.read_csv("traffic_data.csv")
print("Before Cleaning:\n", traffic.describe())
traffic['traffic_density'] = traffic['traffic_density'].fillna(traffic['traffic_density'].mean())
traffic = traffic.drop_duplicates()
traffic['road_name'] = traffic['road_name'].str.lower().str.strip()
traffic['location'] = traffic['location'].str.lower().str.strip()
traffic['speed'] = (traffic['speed'] - traffic['speed'].min()) / (traffic['speed'].max() - traffic['speed'].m
traffic['vehicle_count'] = (traffic['vehicle_count'] - traffic['vehicle_count'].min()) / (traffic['vehicle_co
print("\nAfter Cleaning:\n", traffic.describe())
traffic.to_csv("cleaned_traffic_data.csv", index=False)
```

```
Before Cleaning:
        sensor_id  traffic_density       speed  vehicle_count
count   10.00000         8.000000   10.000000      10.000000
mean     5.50000       127.500000   63.100000     263.000000
std      3.02765       152.877168   24.641654     196.245311
min      1.00000        50.000000   38.000000     140.000000
25%      3.25000        53.750000   45.250000     165.000000
50%      5.50000        67.500000   59.000000     200.000000
75%      7.75000       112.500000   73.500000     270.000000
max     10.00000       500.000000  120.000000     800.000000

After Cleaning:
        sensor_id  traffic_density      speed  vehicle_count
count   10.00000        10.000000  10.000000      10.000000
mean     5.50000       127.500000   0.306098       0.186364
std      3.02765       134.824989   0.300508       0.297341
min      1.00000        50.000000   0.000000       0.000000
25%      3.25000        57.500000   0.088415       0.037879
50%      5.50000        90.000000   0.256098       0.090909
75%      7.75000       125.625000   0.432927       0.196970
max     10.00000       500.000000   1.000000       1.000000
```

**Explanation:**
In this task, traffic data collected from multiple sensors is cleaned and standardized to improve its quality for analysis. Initially, missing values in traffic density are handled using the mean to ensure no gaps in the dataset. Duplicate sensor records are then removed to avoid redundancy and inaccurate analysis. Since the dataset contains inconsistent text formats for road names and locations, these fields are standardized by converting them to lowercase and removing extra spaces. After cleaning, numerical features such as speed and vehicle count are normalized to bring all values into a consistent range, making them easier to compare and analyze.

**Task Description-5**
Social Media Text Data Cleaning and Feature Extraction.
**Expected Output:**
A cleaned text dataset with extracted features suitable for sentiment analysis models.
**Prompt:**
Generate a Python script to clean social media text data by removing duplicates and nulls, cleaning text, tokenizing, removing stopwords, applying stemming, and extracting features like word count and sentiment score.
**CODE AND OUTPUT :**

```python
import pandas as pd
import re
posts = pd.read_csv("social_media_posts.csv")
posts = posts.drop_duplicates(subset='text')
posts = posts.dropna(subset=['text'])
posts = posts[posts['text'].str.strip() != ""]
def clean_text(text):
    text = re.sub(r"http\S+", "", text)   # remove URLs
    text = re.sub(r"[^a-zA-Z\s]", "", text)  # remove special characters
    text = text.lower()  # lowercase
    text = re.sub(r"\s+", " ", text).strip()  # remove extra spaces
    return text
posts['clean_text'] = posts['text'].apply(clean_text)
posts['tokens'] = posts['clean_text'].apply(lambda x: x.split())
stopwords = ['i','this','the','and','is','with','very','not','will','again']
posts['tokens'] = posts['tokens'].apply(
    lambda words: [word for word in words if word not in stopwords])
def simple_stem(word):
    for suffix in ['ing','ed','ly','s']:
        if word.endswith(suffix):
            return word[:-len(suffix)]
    return word
posts['tokens'] = posts['tokens'].apply(
    lambda words: [simple_stem(word) for word in words])
posts['word_count'] = posts['clean_text'].apply(lambda x: len(x.split()))
positive_words = ['love','amazing','excellent','happy','good']
negative_words = ['worst','bad','terrible','not']
def sentiment_score(words):
    score = 0
```

| | clean_text | tokens | word_count | sentiment_score |
|---|---|---|---|---|
| 0 | i love this product | [love, product] | 4 | 1 |
| 1 | worst service ever | [worst, service, ever] | 3 | -1 |
| 2 | amazing experience will buy again | [amaz, experience, buy] | 5 | 0 |
| 3 | check this out great stuff | [check, out, great, stuff] | 5 | 0 |
| 4 | not good very bad experience | [good, bad, experience] | 5 | 0 |
| 7 | excellent quality and fast delivery | [excellent, quality, fast, delivery] | 5 | 1 |
| 8 | terrible support not recommended | [terrible, support, recommend] | 4 | -1 |
| 9 | happy with the purchase | [happy,_purchase] | 4 | 1 |

**Explanation**:
In this task, social media text data is cleaned and prepared for sentiment analysis. Initially, duplicate posts and empty text entries are removed to ensure only meaningful data is processed. The text is then cleaned by removing URLs, special characters, and unnecessary spaces, and converted into lowercase for consistency. After cleaning, the text is split into individual words through tokenization. Common words that do not add much meaning, known as stopwords, are removed to focus on important terms. Basic stemming is applied to reduce words to their root form, helping in better analysis.